支配树算法原理及鸿蒙工具实践

原创 终端平台基础工程 大淘宝技术 2026年1月14日 03:11 浙江 3人

本文介绍了支配树（Dominator Tree）算法在鸿蒙系统 ArkTS 内存分析工具中的应用。为应对淘宝 App 鸿蒙版因内存溢出导致的 Crash 问题，作者构建了一套从客户端采集内存快照、服务端自动分析的工具链。文中对比了多种支配树构建算法（如朴素算法、Lengauer–Tarjan 算法和迭代算法），并说明选用优化后的迭代算法的原因：实现简单、便于验证、且适合存在大量循环引用的内存图结构。

01 前言

随着淘宝App鸿蒙版UV日渐增长，ArkTS的内存溢出导致的Crash也在逐步上升，因此，团队实现了一套从客户端采集内存镜像至服务端自动分析聚合工具。本文介绍该工具的核心原理：内存对象的支配树算法

支配树是什么

在一个有向图中，存在一个入口节点 r，如果节点 u 支配节点 v，当且仅当从 $r \to v$ 的所有路径都经过节点 u，记 $u\ dom\ v$ 如果不存在 u' 使得 $u\ dom\ u'$ 且 $u'\ dom\ v$，则 u 是 v 的直接支配节点，记做 $u\ idom\ v$ 。从入口 r 通过直接支配节点形成的树则称为支配树，即支配树中每个节点的父节点都是其直接支配节点。

支配树作用

在分析内存泄漏或者内存溢出的时候，内存镜像中的对象通常会很多(数量可达百万)。首要的问题是如何找出占用内存较大那些对象，对象占用内存可以分为两类：

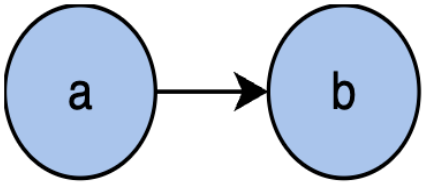
```
class B {
  int c;
}
```

```
class A {
  B b = new B();
}

A a = new A();
```

1. $shallowSize$ ，指的是对象本身的内存大小。上方代码中，a对象中有一个引用指针b，$shallowSize$ 为4bytes。b对象中有一个变量c， $shallowSize$ 为4bytes
2. $retainSize$，指的是回收该对象之后虚拟机能够回收的内存大小。a对象的 $retainSize$ 为8，释放a之后可以同时释放b，所以 $retainSize(a) = shallowSize(a) + retainSize(b)$

显然，只要将那些 $retainSize$ 占用较高的对象回收掉就可以解决内存溢出的问题。如果将对象抽象为有向图中的节点，对象间的引用关系则可以抽象为有向图中的边，例如上面的代码，a对象引用b对象，可以抽象为下图。



因此，内存镜像中的对象集合可以抽象为以GC root为入口节点的有向图，通过有向图构建出对应的支配树。假设在支配树中 a $idom$ $\{b, c, d, ..., n\}$ ，根据支配树的特性，回收 a 对象，$\{b, c, d, ..., n\}$ 在内存中就成了孤立对象可以被GC，由此可以得出：

$$retainSize(a) = shallowSize(a) + retainSize(b) + retainSize(c) + ... + retainSize$$

支配树算法种类

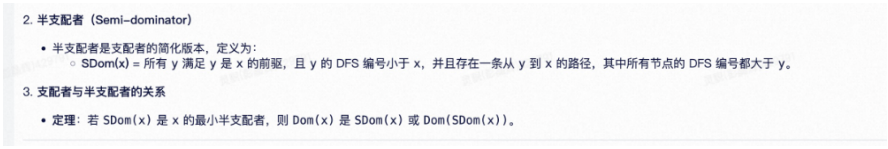
1. 朴素算法，原理是对每个节点 v ，依次删除图中的其他节点，判断是否存在路径从根节点到达节点 v ，算法复杂度为 $O(n^3)$，复杂度过高，一般不使用。
2. Lengauer–Tarjan算法，经典的支配树算法，一般情况是 $O(nlogn)$ 优化后的算法复杂度 $O(n\alpha(n))$ ， α 是阿克曼函数，可以理解为趋近于线性 $O(n)$ 。
3. 迭代算法，通常是用于在编译器的控制流当中，算法复杂度是 $O(n^2)$，优化后算法复杂度 $O(nlogn)$。

实现鸿蒙ArkTS内存镜像分析工具使用的是迭代算法，主要有以下几个原因：

1. Lengauer–Tarjan算法原理及其优化比较复杂，工具类对于构建支配树的速度要求没有那么高；
2. 可以使用华为已实现的profiler插件以及chrome的开源代码作为对比，防止算法出错；
3. 在内存对象的引用关系中，存在很多循环引用，节点与节点之间的环比较严重，这让Lengauer–Tarjan算法在计算半支配节点的时候算法复杂度大大增加。

02 Lengauer–Tarjan算法原理

介绍Lengauer–Tarjan算法的主要原因是网上对于该算法的介绍大部分证明过程都是论文翻译比较复杂，包括有时AI的解答也存在错误。


2. 半支配者（Semi-dominator）
- 半支配者是支配者的简化版本，定义为：
  - SDom(x) = 所有 y 满足 y 是 x 的前驱，且 y 的 DFS 编号小于 x，并且存在一条从 y 到 x 的路径，其中所有节点的 DFS 编号都大于 y。

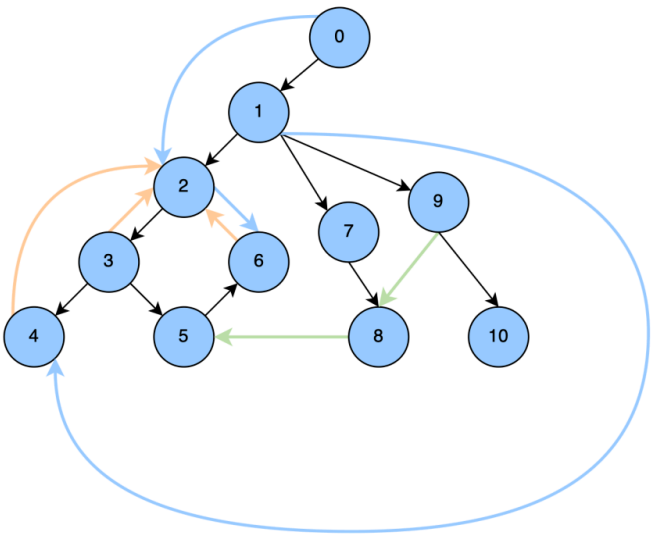
3. 支配者与半支配者的关系
- 定理：若 SDom(x) 是 x 的最小半支配者，则 Dom(x) 是 SDom(x) 或 Dom(SDom(x))。

例如上面是一个AI关于半支配者的定义描述错误，正确的描述应当是所有节点的DFS编号都大于x。

基础知识

下面是一张有序图通过DFS之后形成的节点和边的关系图，节点0–10是dfs遍历的顺序，下面称为dfs序，边的类型可以分为4种：

1. 树边，即DFS过程中建立的边，在下图中显示为黑色；
2. 前向边，即祖先节点到子树非儿子的边，在下图中显示蓝色，2–>6，1–>4，0–>2；
3. 返祖边，即节点到祖先节点的变，在下图中显示为橙色，4–>2,3–>2,6–>2；
4. 横叉边，即从非子树、非祖先横叉过来的边，在下图中显示为绿色，8–>5,9–>8。



理解半支配节点

如何理解半支配节点是理解Lengauer-Tarjan算法关键点，以下是半支配节点 $semi(v)$的定义：

> 对于任意节点 v，如果存在一个节点 $u < v$，且 $u \xrightarrow{x_0 \ldots x_n} v$ 的路径中所有节点 $x_0 \ldots x_n$（x 节点可以不存在)的序号都大于 v，则 u 是 v 的半支配候选，其中dfs序最小的 u 被称为 v 的半支配节点。

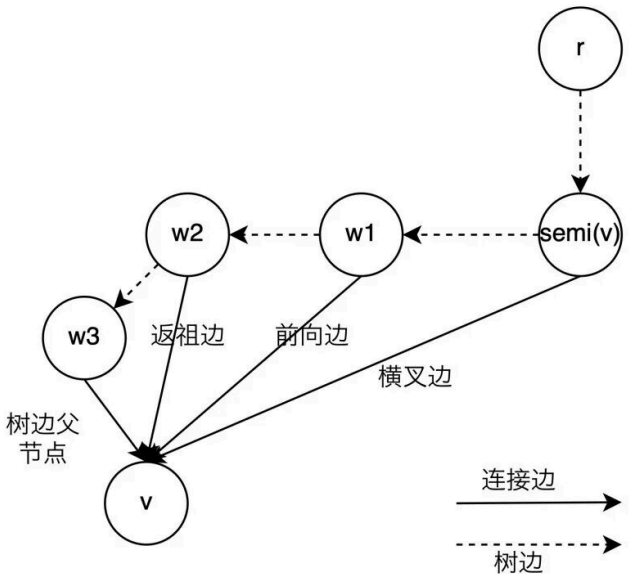
在基础知识中介绍过4种边的类型，对于任意节点 v 的半支配节点可以理解为是通过这四种边到达 v 最远祖先节点(比当前节点dfs序小的节点只有可能是祖先节点或者左子树节点，dfs必须遍历完左子树再遍历右子树，左子树节点不可能存在到右子树节点的边，只可能是祖先节点)。由半支配节点的定义可得：

1. 父节点是半支配候选，例如：3->5
2. 前向边的节点是半支配候选，例如：2->6
3. 返祖边的节点是半支配候选，例如：1->4->2
4. 横叉边的节点是半支配者选，例如：1->7->8->5

换个说法，整个半支配候选的集合是 v 的直接支配者，更进一步，从半支配候选增加一条边指向 v 不改变 v 的支配关系，于是对于任意节点 v，都可以简化为下面的连接关系，其中

$$dfs(r) < dfs(semi(v)) < dfs(w_1) < dfs(w_2) < dfs(w_3) < dfs(v)$$

> 隐含的结论：半支配候选都是在 $semi(v) \rightarrow v$ 的树边上



计算直接支配节点

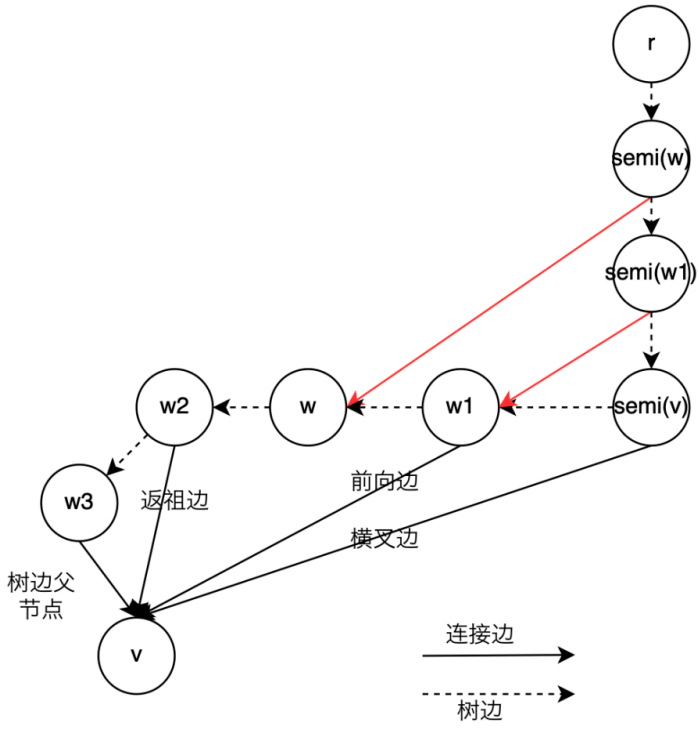
半支配节点 $semi(v)$还不能直接支配节点，因为还不支配 $semi(v) \to v$ 的树边，例如上面的dfs序为5的节点，它存在树边(3->5)和横叉边(1->7->8->5)，通过定义可以算出它的半支配节点 $semi(5) = 1$ 。但实际上还存在0->2->3->5这条路径不经过1号节点，实际上 $idom(5) = 0$。

计算直接支配节点的方式，计算从半支配节点出发经过树边到达 v 节点所有中间节点 w_i 的 $semi(w_i)$ 的最小dfs序 $min(semi(w_i))$ 。

1. $min(semi(w_i)) > semi(v)$ ，则 $idom(v) = semi(v)$
2. $min(semi(w_i)) < semi(v)$ ，假设 $semi(w) = min(semi(w_i))$ ，则 $idom(v) = idom(w)$

在论文中，这两个结论是通过反证法来进行证明，比较难理解，这里尝试通过自然语言来进行证明，不保证完全正确，如有错误，请以论文为准。

1. $min(semi(w_i)) > semi(v)$ 比较好推导，说明不存在有 $semi(v)$ 的祖先节点不通过 $semi(v)$ 到达 w_i ，又由于半支配集合都是处于 $semi(v) \to v$ 的树边上，所以到达半支配集合都得通过 $semi(v)$ ，故 $idom(v) = semi(v)$
2. $min(semi(w_i)) < semi(v)$ 说明至少存在一个节点 $semi(w)$ 不通过 $semi(v)$ 可以到达节点 w_i ，其中 $dfs(semi(w)) < dfs(semi(v))$ ，可以证明 $semi(v)$ 不支配 v ，可以大致得到下图的节点关系，最主要问题是为何不需要考虑其他 $semi(w_i) < semi(v)$ 的情况，例如： $semi(w_1) \to w_1$ 。可以假设 $idom(v) != idom(w)$ ，则存在 $idom(w_i) \to w_i$ 不经过 $idom(w)$ 到达 v ，但是该条路径必须经过 w_i 的半支配候选集，根据前面半支配节点隐含的结论， w_i 的半支配候选集都在 $semi(w_i) \to w_i$ 的树边上，且 $semi(w_i) > semi(w)$ ，由于 $semi(w_i) \to w$ 存在树边到达 w ，这与 $idom(w)$ 的定义相悖，故 $idom(v) = idom(w)$ 。



Lengauer–Tarjan算法实现代码可以参考 Android perflib（cs.android.com/android-studio/platform/tools/base/+/mirror-goog-studio-main:perflib/src/main/java/com/android/tools/perflib/heap/analysis/LinkEvalDominators.kt）

03 迭代算法

算法原理

迭代算法的原理利用了支配节点存在传递性，即 $dom(v)$ 支配节点 v 当且仅当支配其前驱节点 $\{p_i\}$，简要用公式表达:

$$dom(v) = v \cup dom(p_1) \cap dom(p_2) \cap dom(p_3)...dom(p_i)$$

迭代算法过程如下

> 1. 初始化根节点 s，Dom(s) = {s}
> 2. 初始化其他节点 n， Dom(n) = {全集}
> 3. 对于非根节点 n， Dom(n) = n ∪（∩Dom(p)），即当前节点 n 并上所有前驱节点 p的支配节点的交集
> 4. 重复步骤3，直到所有的 Dom(n) 不再变化

JS内存镜像分析

js内存镜像实际上是一个json文件，分为以下几个部分

```
{"snapshot":
{"meta":
{"node_fields":
["type","name","id","self_size","edge_count","trace_node_id","detached
ss","native_size"],
"node_types":
[["hidden","array","string","object","code","closure","regexp","number
"native","synthetic","concatenated
string","slicedstring","symbol","bigint"],"string","number","number","
mber","number","number"],
"edge_fields":["type","name_or_index","to_node"],
"edge_types":
[["context","element","property","internal","hidden","shortcut","weak"
"string_or_number","node"],
"trace_function_info_fields":[...],
"trace_node_fields":[...],
"sample_fields":[...],
"location_fields":[...]
},
"node_count":100056,
"edge_count":384773,
"trace_function_count":0
},
"nodes":[...],
"edges":[...],
"trace_function_infos":[],"trace_tree":[],"samples":[],"locations":[],
"strings":[...],
}
```

- meta: 定义了节点和边的结构
- node_fields: 定义了单个内存节点(node)的属性
- node_types:内存节点(node)的类型
- edge_fields:定义了单个边(edge)的属性
- edge_types: 定义了边的类型
- node_count: 节点数量
- edge_count: 边的数量
- nodes: 节点的集合
- edges: 边的集合
- strings:所有的字符串，包括边的名称，内存节点的名称等等

- 内存节点node

在nodes中，每一行代表一个节点，其中行中的每个数字由node_fields来描述，例如：

```
"node_fields":
["type","name","id","self_size","edge_count","trace_node_id","detachedn
ss","native_size"],

"nodes":[9,11693,1,0,6386,0,0,0
,8,11696,3,48,1,0,0,0
,8,11696,5,48,1,0,0,0
,...]
```

以第一行 `9,11693,1,0,6386,0,0,0` 为例，解析后是这样

```
type: 9, //节点类型的偏移
name: 11693, //节点名字，大概率是文件名，对应strings中的index
id: 1, //无特殊含义，节点的标号
self_size: 0, //大概率是shallow_size，0大概率是数组
edge_count: 6386, //跟其他内存节点的引用关系
```

nodes_type存储了节点的14种类型，这个例子中，节点的类型是synthetic，不过这种类型似乎是虚拟机内部类型，不展示

```
"node_types":
[["hidden","array","string","object","code","closure","regexp","number"
"native","synthetic","concatenated
string","slicedstring","symbol","bigint"],"string","number","number","n
mber","number","number"],
```

strings存储了所有的字符串，包括了节点名字，在这个例子中name转为字符串是SyntheticRoot。

- 引用关系edge

edges中存储了内存节点的引用关系，每一行代表一条边，行中的每一个数字通过edge_fields进行表示

```
"edge_fields":["type","name_or_index","to_node"],

"edges":[5,11673,527080
,5,11673,527072
,5,11673,527056
```

```
,...]
```

type存储的边的类型，也是节点之间引用关系的类型，例如：第一行的例子代表这条边的类型是一个shotcut

```
"edge_types":
[["context","element","property","internal","hidden","shortcut","weak"]
"string_or_number","node"],
context //暂时没有用处
element //DOMTree中的元素
property //类当中的属性
internal //内部属性
hidden //隐藏属性
shortcut //和property中差不多，通过赋值得到的引用
weak //弱引用
```

name_or__index存储的是strings中的名字index，11673对应的是–subroot–字符串to_node代表的是这条边指向的节点索引，因为内存引用构成是一个有向图，所以还需要知道这条边来自于哪个节点。**边属于哪个节点的计算是通过内存节点索引+边来进行计算的，举个例子：如果一个节点的序号是0，有10条边，那么edges[0,10)就是属于节点0的，如果序号1的节点有5条边，那么edges[10,15)就是属于节点1的。**

构建边的索引

内存节点的属性edge_count，代表该节点持有的边的数量，由此可以算出每个节点持有边的索引范围[firstEdgeIndexes[nodeOrdinal], firstEdgeIndexes[nodeOrdinal+1])

```
private void buildEdgeIndexes() {
    firstEdgeIndexes[nodeCount] = edges.length;
    int nodeOrdinal = 0;

    for (int edgeIndex = 0; nodeOrdinal < nodeCount; ++nodeOrdinal) {
        firstEdgeIndexes[nodeOrdinal] = edgeIndex;
        //nodes[nodeOrdinal * nodeFieldCount + nodeEdgeCountOffset]当前
        edgeIndex += nodes[nodeOrdinal * nodeFieldCount + nodeEdgeCount
    }

}
```

构建前驱节点

构建前驱节点的算法比较巧妙

1. 遍历所有的边，找到边的toNode，然后计算toNode的前驱节点(retainer)的数量存储在 `firstRetainerIndex` 中，和firstEdgeIndexes类型，每个节点的前驱节点的范围也计算出来了，就是
[firstRetainerIndex[nodeOrdinal], firstRetainerIndex[nodeOrdinal+1])

```
int edgeLength = edges.length;
for (int edgeToNodeIndex = edgeToNodeOffset; edgeToNodeIndex < edgeLeng
    int toNodeIndex = edges[edgeToNodeIndex];
    if (toNodeIndex % nodeFieldCount != 0) {
        TBPLogger.error("Invalid toNodeIndex is {}", new Object[] {toNo
    } else {
        int toNodeOrdinal = toNodeIndex / nodeFieldCount;
        if (isValid(toNodeOrdinal, firstRetainerIndex.length)) {
            firstRetainerIndex[toNodeOrdinal]++;
        }
    }
}
```

2. 遍历所有的节点，然后为前驱节点预留好位置，retainingNodes是前驱节点的集合

```
int firstUnusedRetainerSlot = 0;
for (int nodeOrdinal = 0; nodeOrdinal < nodeCount; ++nodeOrdinal) {
    int retainerCount = firstRetainerIndex[nodeOrdinal];
    firstRetainerIndex[nodeOrdinal] = firstUnusedRetainerSlot;
    if (isValid(firstUnusedRetainerSlot, edgeCount)) {
        retainingNodes[firstUnusedRetainerSlot] = retainerCount;
        firstUnusedRetainerSlot += retainerCount;
    }
}
```

3. 遍历所有的节点，再根据节点持有的边找到toNode，那么该节点就是toNode的前驱节点

```
for (srcNodeOrdinal = 0; srcNodeOrdinal < nodeCount; ++srcNodeOrdinal)
    firstEdgeIndex = nextNodeFirstEdgeIndex;
    nextNodeFirstEdgeIndex = firstEdgeIndexes[srcNodeOrdinal + 1];
    srcNodeIndex = srcNodeOrdinal * nodeFieldCount;

    for (int edgeIndex = firstEdgeIndex; edgeIndex < nextNodeFirstEdge
        int index = edgeIndex + edgeToNodeOffset;
```

```
        if (isValid(index, edges.length)) {
            int toNodeIndex = edges[index];
            if (toNodeIndex % nodeFieldCount != 0) {
                TBPLogger.error("Invalid toNodeIndex is {}.", new Obje
            } else if (isValid(toNodeIndex / nodeFieldCount, firstReta
                //retainningNodes的起点
                int firstRetainerSlotIndex = firstRetainerIndex[toNode
                //由后向前添加，直到retainingNodes[firstRetainerSlotIndex
                int nextUnusedRetainerSlotIndex = firstRetainerSlotInd
                if (isValid(nextUnusedRetainerSlotIndex, retainingNode
                    retainingNodes[nextUnusedRetainerSlotIndex] = srcN
                    retainingEdges[nextUnusedRetainerSlotIndex] = edge
                }
            }
        }
    }
}
```

计算节点distance

计算节点就是使用bfs进行遍历

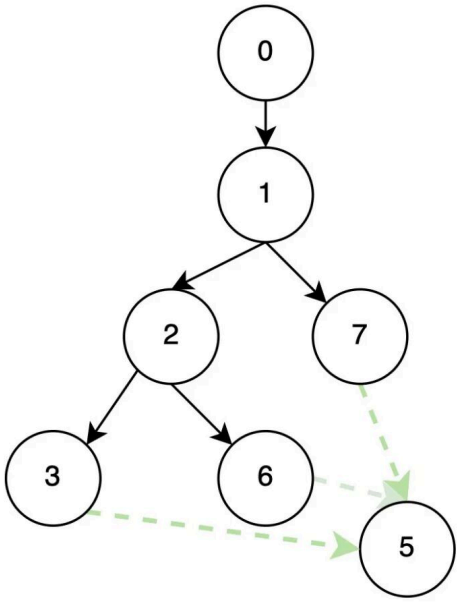
```
private void calculateDistances() {
    for (int nodeOrdinal = 0; nodeOrdinal < nodeCount; nodeOrdinal++)
        nodeDistances[nodeOrdinal] = noDistance;
    }
    int[] nodesToVisit = new int[nodeCount];
    int nodesToVisitLength = 0;
    //遍历根节点
    for (int edgeIndex = firstEdgeIndexes[rootNodeIndexInternal];
         edgeIndex < firstEdgeIndexes[rootNodeIndexInternal + 1];edgeI
        int childNodeIndex = edges[edgeIndex + edgeToNodeOffset];
        int childNodeOrdinal = childNodeIndex / nodeFieldCount;
        if (isUserRoot(childNodeIndex)) {
            nodeDistances[childNodeOrdinal] = 1;
            bfsNodes[bfsIndex++] = childNodeOrdinal;
            nodesToVisit[nodesToVisitLength++] = childNodeIndex;
        }
    }
    //从根节点进行遍历
    bfs(nodesToVisit, nodesToVisitLength, nodeDistances);
    nodeDistances[rootNodeIndexInternal / nodeFieldCount] =nodesToVisi
    nodesToVisit[0] = rootNodeIndexInternal;
```

```
    nodesToVisitLength = 1;
    //遍历其他单独的节点，synthetic节点
    bfs(nodesToVisit, nodesToVisitLength, nodeDistances);
}
```

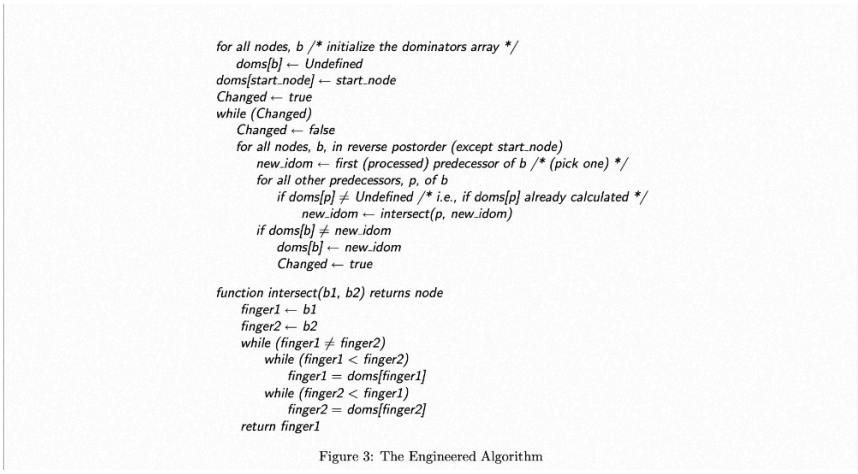
构建支配树

chrome的开发者工具和鸿蒙IDE的profiler插件使用的都是优化过的迭代算法，参考论文《A Simple, Fast Dominance Algorithm》，有两个重要的优化点：

1. 计算节点的Dom顺序不是采用先序遍历，而是采用逆后序进行遍历加速收敛。**从基础知识那里可以得知dfs树有四种边，其中前向边和树边，先序遍历和逆后序遍历都是一次遍历就可以计算，对于返祖边，都是需要多次遍历才能计算，但对于横叉边，逆后序只需要一次遍历而先序遍历就要多次。**
2. 在计算 Dom(p) 的交集的朴素算法是对所有前驱节点的支配节点一一查找匹配，这样的效率很低很低，甚至可以到达 $O(n^2)$ 。**论文中提出了一个新的方案，由于前驱节点的dom树已经构建完成，例如下图dom(7) = {7,1,0}, dom(6) = {6,2,1,0}， dom(3) ={3,2,1,0}，此时求dom(5)，通过观察很容易发现dom(5)={5,1,0}。根据树的特点，求解 idom(n)就转换为求解n的前驱节点在支配树上的公共祖先节点。**



这里给出论文中的伪代码方便理解。具体实现代码可以参考chrome的代码HeapSnapshot.ts（github.com/ChromeDevTools/devtools-frontend/blob/chromium/6194/front_end/entrypoints/heap_snapshot_worker/HeapSnapshot.ts）



```
for all nodes, b /* initialize the dominators array */
    doms[b] ← Undefined
doms[start_node] ← start_node
Changed ← true
while (Changed)
    Changed ← false
    for all nodes, b, in reverse postorder (except start_node)
        new_idom ← first (processed) predecessor of b /* (pick one) */
        for all other predecessors, p, of b
            if doms[p] ≠ Undefined /* i.e., if doms[p] already calculated */
                new_idom ← intersect(p, new_idom)
        if doms[b] ≠ new_idom
            doms[b] ← new_idom
            Changed ← true

function intersect(b1, b2) returns node
    finger1 ← b1
    finger2 ← b2
    while (finger1 ≠ finger2)
        while (finger1 < finger2)
            finger1 = doms[finger1]
        while (finger2 < finger1)
            finger2 = doms[finger2]
    return finger1
```

Figure 3: The Engineered Algorithm

计算支配节点的核心就在intersect方法中，找当前节点的dom和前驱节点之间的最近公共祖先，优化后的迭代算法可以降低到 $O(m*nlogn)$ ，m是遍历次数logn是支配树的深度，更进一步，如果m是一个常量的话，一般算法复杂度可以做到 $O(nlogn)$ 。

计算retainSize

retainSize就是如果通过GC释放这个内存节点，能释放的内存大小。计算支配树的目的就是确定节点的retainSize = 当前节点的shallowSize(自身大小) + 支配节点的retainSize。

```
1  void calculateRetainedSizes(int[] postOrderIndex2NodeOrdinal) {
2      int postOrderIndex;
3      for (postOrderIndex = 0; postOrderIndex < nodeCount;++postOrderInd
4          retainedSizes[postOrderIndex] = nodes[postOrderIndex *nodeFiel
5          if (nodeNativeSizeOffset != 0) {
6              retainedNativeSizes[postOrderIndex] = (long)nodes[postOrde
7          }
8      }
9      //后序遍历，让被支配节点先于支配节点计算retainSize
10     for (postOrderIndex = 0; postOrderIndex < nodeCount - 1;++postOrder
11         int nodeOrdinal = postOrderIndex2NodeOrdinal[postOrderIndex];
12         int dominatorOrdinal = 0;
13         if (isValid(nodeOrdinal, dominatorsTree.length)) {
14             dominatorOrdinal = dominatorsTree[nodeOrdinal];
15         }
16
17         if (isValid(dominatorOrdinal, retainedSizes.length) &&isValid(
18             retainedSizes[dominatorOrdinal] +=retainedSizes[nodeOrdina
19             if (nodeNativeSizeOffset != 0) {
20                 retainedNativeSizes[dominatorOrdinal] +=retainedNative
21             }
22         }
23     }
```

```
24
25 }
```

聚合计算

虽然最难的部分支配树已经计算完成，但还需要将引用链聚合起来，方便人工分析和上报到平台进行自动化分析。

具体流程如下：

1. 先遍历所有节点，拿到所有节点的stringIndex，根据stringIndex进行聚合，可以得到对象类型的retainSize和个数，然后进行过滤掉一些泄漏不多的对象类型，得到大对象类型。

```
1  public Map<Integer, ConstructorNode> getBigHeapNode() {
2      Map<Integer, ConstructorNode> heapNodes = new HashMap<>(nodeCount)
3      for (int nodeOrdinal = 0; nodeOrdinal < nodeCount; nodeOrdinal++)
4          int nodeIndex = nodeOrdinal * nodeFieldCount;
5          if (isHidden(nodeIndex)) {
6              continue;
7          }
8          if (isSynthetic(nodeIndex)) {
9              continue;
10         }
11         int stringIndex = nodes[nodeIndex + nodeNameOffset];
12         if (heapNodes.containsKey(stringIndex)) {
13             ConstructorNode heapNode = heapNodes.get(stringIndex);
14             heapNode.addRetainedSize(1, retainedSizes[nodeOrdinal]);
15         } else {
16             String constructorName = strings.get(stringIndex);
17             if (HarmonyAnalyzeUtils.containsTaobao(constructorName)) {
18                 ConstructorNode heapNode = new ConstructorNode(stringI
19                 heapNodes.put(stringIndex, heapNode);
20             }
21
22         }
23     }
24     ConfigParams configParams = CmdArgs.getInstance().getConfigParams(
25     Map<Integer, ConstructorNode> bigNodes = new HashMap<>();
26     for (ConstructorNode heapNode : heapNodes.values()) {
27         if (heapNode.getRetainedSize() >= configParams.minRetainedSize
28             bigNodes.put(heapNode.getStringIndex(), heapNode);
29         }
30     }
31     return bigNodes;
```

```
}
```

2. 重新通过bfs的顺序遍历节点，如果是大对象类型就进行回溯，找到到根节点的路径并记录下来，最后进行根据引用路径进行聚合成aggregatedRefTree。

```
private void aggregateReferenceTree() {
    Map<Integer, ConstructorNode> bigNodes = getTBHeapNode();
    //对节点进行bfs
    for (int i = 0; i < bfsIndex; i++) {
        int nodeOrdinal = bfsNodes[i];
        int nodeIndex = nodeOrdinal * nodeFieldCount;
        int stringIndex = nodes[nodeIndex + nodeNameOffset];
        if (bigNodes.containsKey(stringIndex)) {
            Map<String, List<ConstructorNode>> referenceTrees = new Ha
            calculateReferenceTree(nodeOrdinal, new StringBuilder(), n
            for (String key : referenceTrees.keySet()) {
                List<ConstructorNode> refChain = referenceTrees.get(ke
                if (aggregatedRefTree.containsKey(key)) {
                    ConstructorNode cached = aggregatedRefTree.get(key
                    //相同节点的链路不用相加
                    if (cached == refChain.get(0)) {
                        continue;
                    }
                    cached.addCount();
                    cached.addRetainedSize(retainedSizes[nodeOrdinal])
                    cached.addShallowSize(nodes[nodeIndex + nodeSelfSi
                } else {
                    if (!refChain.isEmpty()) {
                        aggregatedRefTree.put(key, refChain);
                    }
                }
            }
        }
    }
}
private void calculateReferenceTree(int nodeOrdinal, StringBuilder pat
                                    Map<String, List<ConstructorNo
    int nodeIndex = nodeOrdinal * nodeFieldCount;
    int stringIndex = nodes[nodeIndex + nodeNameOffset];
    pathKey.append(stringIndex).append('-');
    ConstructorNode node = new ConstructorNode(stringIndex, strings.ge
    refChain.add(node);
    if (nodeDistances[nodeOrdinal] == 1) {
        String key = pathKey.toString();
```

```
            List<ConstructorNode> newRefChain = new ArrayList<>(refChain);
            referenceTree.put(key, newRefChain);
            return;
        }
        List<Integer> retainers = findRetainers(nodeOrdinal);
        for (Integer retainer : retainers) {
            StringBuilder newPathKey = new StringBuilder(pathKey);
            List<ConstructorNode> newRefChain = new ArrayList<>(refChain);
            calculateReferenceTree(retainer, newPathKey, newRefChain, refe
        }
    }
```

04 总结

支配树算法属于是内存泄漏/溢出分析的核心算法，我们基于此算法实现鸿蒙ArkTS内存镜像分析工具，希望本文对大家实现类似功能的工具有一定帮助。

05 参考文档

- 《A Simple, Fast Dominance Algorithm》：https://www.cs.tufts.edu/~nr/cs257/archive/keith-cooper/dom14.pdf
- 《A New, Simpler Linear-Time Dominators Algorithm》：http://adambuchsbaum.com/papers/dom-toplas.pdf
- 《A Fast Algorithm for Finding Dominators in a Flowgraph》：https://faculty.cc.gatech.edu/~harrold/6340/cs6340_fall2009/Readings/lengauer91jul.pdf

06 团队介绍

本文作者灵貌，来自淘天集团-终端平台基础工程团队，团队负责淘宝App基础架构和基础链路业务，致力于通过技术创新和系统优化，打造一个更稳定、更流畅的手机淘宝。同时我们也在持续探索通过AI搭建高效、智能的研发体系，提升团队的研发效能。

¤ 拓展阅读 ¤

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法



大淘宝技术

大淘宝技术官方账号

967篇原创内容

公众号

阅读 1634

留言

写留言

还没有留言，来写首评